

JavaScript Unit Testing

Dumitru Andreea-Alexandra, 344
Nedelcu Ionut-Daniel, 342

TABLE OF CONTENTS

I. Application	A tour of StudyBuddies, the team-management app we wrote and used as our system under test
II. Equivalence Class Partitioning	How we group inputs into classes so a handful of representative cases stand in for thousands of similar ones
III. Boundary Value Analysis	Pushing every numeric or length-bounded field to its edges, where bugs love to live
IV. Category Partitioning	Combining input categories with a one-valid-at-a-time approach, and pruning cases that collapse to the same outcome
V. Control Flow and Coverage Metrics	How we used Jest --coverage to measure statements, branches, conditions and independent paths through every mutation
VI. Mutation Testing	How Stryker breaks the source on purpose, and how every change we made ended up being noticed by the test suite
VII. AI Report	What we learned or observed while trying to generate a test suite only using prompts, without any manual coding (see the AI Report for objective data)
VIII. Conclusions	The final numbers from the coverage and mutation reports, plus a few honest takeaways

I. Application

StudyBuddies → small team-management platform we wrote and used as the system under test.

The point of the project → real, non-trivial code with branches, validations and authorization.

Backend stack → Express 5 (&GraphQL), Sequelize (&SQLite), JWT (auth), bcrypt (passwords).

Domain → 12 Sequelize models built around Users, Roles, Projects, Sprints, Tasks, Comments, Teams, Positions and Repositories.
→ the schema is split across 13 GraphQL types, 16 input types and 30 mutations.

Roles and access → Three roles: Admin, Manager and Employee (each with a list of allowed actions).

What we tested → The 16 mutations that change state.
→ register, login, role assignment, project CRUD plus user-project links, sprint CRUD, and the full task lifecycle (create, update, delete, assign, change status).

II. Equivalence Class Partitioning

- Idea** → if two inputs trigger the same rule, the program should react the same way (one test per class)
- each field gets one valid class, and the invalid ones we could think of (e.g. missing, too short, too long, wrong format, already in use).
 - we picked a representative value out of every class (it becomes the input of one Jest test case).

Walk-through: register - email field

- six classes: valid format (e.g. "user@studybuddies.com"), empty string, length below 5, length above 254, bad regex (e.g. "invalid-email"), and a duplicate that is already in the seed data.

How it scales to the rest of the app

- same recipe for users (username, password, names), projects (name, description, repository link), sprints (number, dates, project) and tasks (status, assignee, project, sprint).

Why we like it

- it forces us to read the validation code one rule at a time, so the test suite mirrors the real branches in the source instead of duplicating cases that would never tell us anything new.

III. Boundary Value Analysis

Boundary value analysis → the natural follow-up to ECP
→ bugs love to hang around the edges, so we test the values right next to them.
→ for every numeric or length-bounded field we wrote four cases: just below the minimum, exactly the minimum, exactly the maximum and just above it.

Users → Email length 4 / 5 and 254 / 255. Password length 7 / 8 and 64 / 65. Username 2 / 3 and 30 / 31.
First and last name 1 / 2 and 50 / 51.

Projects → Project name 2 / 3 and 50 / 51 characters. Description 500 / 501, plus the existence boundary on repository and project IDs.

Sprints → sprint number 0 / 1, sprint description 2000 / 2001 and the date order rule where start must be strictly before end.

Tasks → Tasks name 200 / 201, description 2000 / 2001 and the allowed status set Open / In Progress / Done / Closed.

BVA reuses the same fixtures as ECP, so the extra confidence costs us very little.

IV. Category Partitioning

Category partitioning → glues ECP and BVA together for a real-world mutation with many fields at once.
→ For each GraphQL mutation we listed its input categories: caller context, every required field, every optional field and the relevant database state.
→ Each category got a list of representative values.

One-valid-at-a-time → a test case is built by combining one value from each category. We keep every category valid except one, so when a test fails it points to a single rule rather than a tangle of issues.

Pruning → when two choices map to the same outcome we merge them.
(e.g. in createProject both a missing description and a short valid one return success, so we keep just one. Same for the optional repository link.)

A concrete list, not a vague label

- for createSprint, the categories were context, sprintNumber, description, startDate, endDate and projectID.
- from those we got 10 cases: AllValid, NonAdmin, NumberMissing, NumberZeroOrNegative, NumberDuplicate, NumberNonInteger, DescriptionTooLong, StartInvalidFormat, EndInvalidFormat and ProjectNotFound.

V. Control Flow and Coverage Metrics

Once we had the tests in place → we ran them with Jest --coverage
→ the runner could keep score on every line of source code that actually executed.

Statement coverage → did every executable statement run at least once?
→ we made sure each success path and each error message was hit by some test.

Branch and condition coverage → did every if, else and ternary go both ways?
→ did each sub-condition (length, format, uniqueness, existence) flip true and false at least once?

Independent paths → for each mutation we kept a minimal set of execution paths.
→ one for the success case and one per unique error exit.
→ path diagrams are stored under “./_diagrams”.

How we got there → we started from a few tests, looked at the HTML report under “./coverage” to see the lines coloured red.
→ added one case at a time until everything turned green.

VI. Mutation Testing

Coverage → tells us if our tests touch every line, but it does not tell us whether they would catch a bug.
→ mutation testing solves that by editing the source on purpose and asking the same question: do the tests still fail?

How we ran it → Stryker is configured in stryker.conf.js and started with "npm run test:mutation".
→ it uses our existing Jest project and reports under "./reports/mutation/".

What it mutates → all user, project, sprint, and task GraphQL mutations
→ plus "utils/authorize.js" and the JWT middleware.
→ operators include arithmetic, comparison, logical, string and conditional swaps.

Results → mutation score: 100% - 958 detected out of 958 valid mutants.
→ no surviving mutants means no easy patch the code could have absorbed without our tests noticing.
→ that is the strongest signal we got that the test suite is doing real work.

VII. AI Report

The approach → we wanted to see how an AI coding agent would work by itself to build a complete testing suite, without having any access to our already existing suite.

→ applying a comparison approach, we did not want to interact or give too much input to the model (the prompts and responses are available in the AI Report)

Did it work? → kind of. Even though it obtained a 100% testing coverage and 95% mutant coverage, the 5% gives enough space for eventual attackers to take over the app in a way or another.

→ one of the mutants that timed out (and didn't die) was in "registerMutation". The problem was related to how the "bcrypt" algorithm works, especially when it comes to mutations.

Thoughts? → although we have seen instances when AI fails miserably in development, when it comes to testing, it can brush off the initial repetitive test writing.

→ agents need to be supervised, especially in corner cases, when handling sensitive information, complex code structures or not given enough context.

Could it have performed differently?

→ if given only a specific script from a bigger project, without any understanding of how the file is tied to others, AI agents would perform badly

→ if given clearer instructions and more code review iterations, AI agents would perform a lot better.

Conclusions

1

FULL COVERAGE

Jest reports **100%** on statements, branches, functions and lines (442/442 - 321/321 - 19/19) across 230 tests, with no red lines left in the HTML report.

2

MUTATION SCORE

Stryker generated **1032 mutants**. 958 killed, 0 survived, 0 timeout and 74 ignored static - a **100%** mutation score on the 958 valid ones.

3

STRATEGIES WORK

ECP gave us the rules, **BVA** pinned down the edges and category partitioning kept combinations under control - together they produced a **small but sharp suite**.

4

LESSONS LEARNED

Starting from a few tests and **adding cases** category by category scales **better than guessing**. Reading the coverage and Stryker reports drove most of our progress.